Summer 2026 Induction Assignment

Society for Artificial Intelligence and Deep Learning

www.saidl.in

This is the official assignment for SAiDL inductions. Please join our Slack Workspace (invite) and fill in this form to register.

The deadline is Saturday, May 16, 2026 at 11:59 PM IST.

A Note from Us

Through this assignment, our aim is to introduce you to the key and upcoming ideas in Machine Learning, Deep Learning, and Artificial Intelligence. We value your time and have designed each question such that the process of solving it will add to your skill set while also giving us an idea of your abilities. We have also provided a collection of resources to help you learn the tools and concepts required along the way.

Please, go through the instructions before attempting the questions. The questions may seem intimidating at first glance. Don't panic! They are meant to be challenging and will take some time and effort to complete. Try looking around to understand the concepts involved in the questions and try digging into these topics. It is expected that around **10% - 40%** of the time (depending on your previous familiarity) on each question will be spent **learning** new concepts. Submissions will be judged primarily on the basis of the **approach** and the **thought** that went into attempting the answer, so be sure to document everything. Also, make sure to submit your attempt regardless of whether or not you have completed the assignment as a whole.

If you have any doubts, **try to search** for solutions on the internet first before asking doubts on the slack. We will also be assigning everyone mentors after the second week to stay updated with your progress.

All the very best!

Submission Guidelines

The submission deadline is **Saturday 16/05/2026 at 11:59 PM IST**.
Submission form: To be shared later.

The submission should be in the form of a Github repository titled **SAiDL-Summer-Assignment-2026**. Your repository must be **kept private up until the submission deadline**, but don't forget to make the repo public when submitting the assignment.

You are required to document your approach and results for each question you attempt and analyses wherever required, in detail. The documentation should be included in the GitHub repository along with the code. The documentation must be made in LaTeX. Make sure your final report for each task is as detailed and clear as possible. You are free to use other code bases, but ensure that the **tasks** in the given question have been **fulfilled**.

Any form of **plagiarisation** or **collaboration** with others is strictly prohibited and will be penalized.

You'll be allotted a mentor for the assignment **after the first two weeks** and you are required to inform them whenever you complete a question (submit the Google form only after you have completed the assignment or at the deadline). All of this will be done through our Slack Workspace so make sure you join it to receive all the assignment related updates and for asking queries that you might have. You can also contact those mentioned below for queries.

SAiDL holds the rights to the questions asked in the assignment. If you choose to use this outside the assignment, you need to acknowledge/cite SAiDL.

Contact Details

In case of any doubts, clarifications, or guidance, you can contact one of us. We request that you stick to Slack as the preferred mode of communication for all the questions that you have as it will also benefit others. However, you can reach out to us through other means in case we fail to respond on Slack.

- Harshvardhan Mestha (Diffusion and Core ML) - +91-8334825566
- Sasmit Datta (Diffusion and Core ML) - +91-8334825566
- Aditya Nagarsekar (Sparsity & Optimization) - +91-8879845271
- Soham Kalburgi (Sparsity & Optimization) - +91-8329911228
- Pratham Chheda (Diffusion and RL) - +91-6399639934
- Anish Sathyanarayanan (Mechanistic Interpretability) - +91-9150265302
- Aarush Rathore (Mechanistic Interpretability) - +91-8815983629
- Saatwik Tiwari (RL) - +91-9045330144
- Kamal Manchanella (Core ML) - +91-7680830711

Resources

The following is a list of resources for tools and topics you will need to be familiar with in order to solve the assignment questions. This section is more a selection of resources for things you will need along the way. It is **not mandatory** to complete, you should treat it more as a roadmap of things to learn and as a reference to get back to.

- ***Python***
  Python is the Lingua Franca of AI (at the moment). Check out this guide by Google to start or the tutorial series by Sentdex or Corey Schafer. These books [1], [2] and the official documentation are good references.

- ***Coursera Specialization on Machine Learning***
  This specialization is the launchpad for most AI enthusiasts and gives you a first hands-on approach to Machine Learning along with the math behind it.

- ***Linux Terminal***
  This is one of the fundamental requirements for **any** computer scientist. The following will help you get started: [1], [2].

- ***Numpy***
  Crudely speaking this is **MATLAB** for Python. You can go to its official tutorials (we recommend version 1.x as 2.x is not as widely adopted yet), this is a library that will become easy to operate once you start using it. Good luck with shape errors!

- ***Pandas***
  This is one of the **most** crucial and powerful libraries in data science. To begin with, you end up learning how to read and write CSV and JSON files as well as how to manipulate Data Frame rows, columns, and contents. Again Sentdex to the rescue.

- ***Matplotlib***
  Learn how to plot basic graphs. The pyplot submodule **should be enough** for the beginning. Sentdex is your saviour again. The docs are useful too!

- ***PyTorch***
  PyTorch is a Machine Learning **framework** built to provide a toolkit for writing Deep Learning research and application code. We would suggest that you get hands-on experience with PyTorch by following the official tutorials. These tutorials by Patrick Loeber also help.

- ***Git***
  An integral part of most software projects, Git is a tool for **version control** and managing changes to code. This is a cool guide for a quick overview, for a more detailed intro check out this course. Make sure to sign up for the Student Developer Pack on GitHub.

- ***Conda***
  Managing different packages when you are working on multiple projects can be a pain (especially in Python). Conda (and various other similar tools) is a highly featured **package manager** which makes life much easier. Check it out at the docs and learn how to use it here or here.

- ***ML and AI***
  For Core Machine Learning, we recommend Bishop's "Pattern Recognition and Machine Learn-

ing" Textbook [Link] and for Core AI, Russel and Norvig's "Artificial Intelligence: A Modern Approach" [Link]. Both of these are very **comprehensive** textbooks that cover the fundamentals of both areas with easy to understand explanations. You **do not** need to complete them cover to cover in one go, but can use them more as reference books, learning about specific topics at a time.

For those who are new to Deep Learning, we recommend you start with Stanford's cs231n (Deep Learning for Computer Vision): [YouTube Link] [Course Link] course which covers everything from loss functions and basics of Neural Networks to Computer Vision. Go through the video lectures and **attempt** the accompanying assignments (we recommend doing assignments from the latest offering of the course as well) to get a good idea of various fundamental concepts. The Deep Learning Book [Link] is also a great resource for detailed introductions to a wide range of concepts.

We also recommend a parallel from Harvard, [CS 197]. It has detailed notes on the implementation and coverage of more recent concepts.

- ***Overleaf and Prism***
  For creating your LaTeX report, we highly recommend using Overleaf or Prism, both user-friendly editors for working with LaTeX files. If you're new to these platforms or LaTeX in general, now is the perfect time to get started! Learning to create LaTeX documents is a valuable skill to have. You can explore the official Overleaf tutorials or check out this helpful YouTube playlist designed for beginners.
  PS: This document itself was made using Prism.

- ***Weights & Biases***
  For tracking your experiments, we recommend using Weights & Biases, a centralized platform for logging hyperparameters, monitoring real-time performance metrics (like loss and accuracy), and visualizing model gradients to streamline the iterative development process.

- ***Hydra***
  For configuration of your experiments, we recommend using Hydra, a flexible framework for dynamically composing hierarchical configurations, easily overriding parameters directly from the command line, and managing complex hyperparameter sweeps to ensure clean, modular, and reproducible setups.

Assignment

The assignment is made up of the following sections:

1. Core ML Task (**Compulsory**)
2. Domain-Specific Tasks (**Any ONE out of 4**)
   (a) Diffusion
   (b) Mechanistic Interpretability
   (c) Reinforcement Learning
   (d) Sparsity & Optimization
3. Paper Implementation

You have to attempt this assignment in **one** of the following ways:

1. Core ML ***and*** one of the four domain-specific tasks.
2. Paper Implementation task **only**.

Every question has a **BONUS** section. This is only to be done after the preceding tasks for that question have been completed.

Please note that you will be judged primarily on the effort you have put in and your understanding of the concepts involved.

Along with how well you perform the tasks, you will also be evaluated on how clearly and thoroughly you **document** your approach and results. So make sure to make your final **LaTeX report** for each task as detailed and clear as possible.

Core ML

Long-context modeling remains a core challenge in modern sequence models due to the quadratic complexity of vanilla self-attention and limitations in positional extrapolation. Recent architectural advances propose modified attention mechanisms, alternative positional encodings, and hybrid convolutional/sequence modeling approaches to improve efficiency and long-range dependency capture.

In this assignment, you will design a modular long-context sequence model and conduct a systematic empirical study.

Dataset & Task: All experiments will be grounded in a sequence modeling task using the *WikiText-2* dataset (or a subset depending on your compute).

Base Model: Your starting baseline must be a standard Transformer architecture, but you are free to determine its specific size and hyperparameters (e.g., number of layers, embedding dimension, number of heads) based on your available compute.

Your code must be structured such that attention types, positional encodings, and architectural blocks can be swapped independently.

1. **Basic Transformer Baseline**
   (a) Implement a standard Transformer sequence model (as in *"Attention Is All You Need"*).
   (b) Train on WikiText-2 using a context window of length 1024 tokens. Establish your baseline validation perplexity and throughput.
2. **Attention Variants**
   (a) Implement and evaluate at least three of the following to replace the standard self-attention mechanism: Sliding Window/Local Attention, Sparse Block Attention, Linear Attention (Kernelized/Performer-style), Multi-Query Attention (MQA) / Grouped Query Attention (GQA), or Softmax-free attention (e.g., Kernel/ReLU-based normalization).
   (b) Evaluate across multiple context lengths (e.g., 512, 1024, 2048, and 4096 if feasible).
   (c) **Measure:** Validation perplexity, peak GPU memory consumption, training stability, and inference latency (tokens/sec).
3. **Positional Embeddings**
   (a) Replace your baseline's standard positional encoding with the following variants: Rotary Positional Embedding (RoPE), ALiBi (Attention with Linear Biases), and Relative Positional Encoding.
   (b) Optionally, implement Positional Interpolation/Scaling for context extension.
   (c) **Extrapolation Test:** Train each model on $L_{train} = 512$. Evaluate validation perplexity on $L_{test} = 512$, $L_{test} = 1024$, and $L_{test} = 2048$ to measure how well the encodings extrapolate beyond their training length.
4. **Convolution + Attention Hybrids**
   (a) Modify the architecture to include a 1D convolutional component to efficiently capture local n-gram context.

(b) Implement at least two designs: Conv1D layer before each attention block, Interleaved Conv + Attention blocks, replacing a subset of attention layers with depthwise separable 1D convolutions, or Gated convolutional feedforward networks.

(c) Evaluate the resulting architectures combined with your best attention variant (Part 2) and best positional encoding (Part 3), comparing the impact on downstream perplexity vs. compute.

5. **Logging and Reporting**

   (a) For all experiments, report training and validation loss, validation perplexity, peak GPU memory usage (in MB/GB), training time per epoch, and throughput (tokens/sec). Present your findings in a comparative table.

BONUS

The Attention-Free Transformer (AFT) is an efficient alternative to standard self-attention that replaces explicit attention matrices with query-gated, position-biased aggregation. This suggests a broader design space of *attention-free* Transformer variants that may offer improved efficiency, scalability, or inductive bias.

Implement the AFT variants mentioned in the paper, "An Attention Free Transformer" and compare it with the best performing version from the previous tasks. If you're up to the challenge, you can also try to find genuine gaps in the AFT paper's method and design your own AFT variant, implement it and benchmark it similarly, but your method should be well justified for all changes.

Resources:

- Attention Is All You Need
- Longformer: The Long-Document Transformer
- Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention
- Fast Transformer Decoding: One Write-Head is All You Need
- GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints
- Rethinking Attention with Performers
- Self-Attention with Relative Position Representations
- RoFormer: Enhanced Transformer with Rotary Position Embedding
- Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation
- Conformer: Convolution-augmented Transformer for Speech Recognition
- An Attention Free Transformer

Recommended Platform: Google Colab, Kaggle Notebook

Diffusion

Diffusion models generate high-quality images by iteratively denoising a noise-corrupted signal. The Diffusion Transformer (DiT) replaces the traditional CNN-based U-Net backbone with a scalable Transformer, enabling better image fidelity at higher resolutions. While these models excel globally, complex or fine-grained regions often demand computationally expensive *cyclic refinement* (re-injecting noise and re-denoising). Fixed spatial heuristics like Region-Aware Sampling (RAS) apply this refinement uniformly, ignoring per-image semantics. To remedy this, you will implement *Regionally-Adaptive Cyclic Diffusion (RACD)*: train a lightweight "Difficulty Predictor" that spatially identifies which latent regions need cyclic refinement, maximising fidelity while minimising compute.

Dataset: Train on the landscape dataset (no class conditioning).

1. **Baseline Global Cyclic Refinement**
   Cyclic refinement is a technique that improves sample quality by re-injecting a small amount of noise into already-denoised latents and running additional denoising steps. This acts as a "polish" pass, correcting residual errors at the cost of extra compute.

   (a) Implement a global cyclic refinement step on your baseline DiT trained on the landscape dataset. After standard sampling reaches $t = 0$, inject noise back into the latents up to $t = 200$, then denoise back to $t = 0$.

   (b) Measure average generation time and FID/CMMD. Do you notice a visual improvement? Explain the compute trade-off.

2. **The Difficulty Predictor**
   Rather than applying cyclic refinement everywhere (expensive) or nowhere (lossy), a difficulty predictor identifies *which spatial regions* genuinely need it. The predictor is supervised using the discrepancy between an intermediate x_0 prediction and the final ground-truth x_0 as a proxy for "how hard" each region is to denoise.

   (a) Design and train a lightweight spatial predictor network (e.g., a shallow CNN or linear projection on intermediate DiT features) to output a spatial difficulty map $M \in [0, 1]^{h \times w}$. Use the variance/error between the intermediate predicted x_0 and the final ground-truth x_0 during the initial sampling pass to supervise which regions require refinement.
   Task split: First train your base DiT to convergence on the landscape dataset. Then freeze the DiT weights and train only the Difficulty Predictor on top of the frozen DiT's intermediate features.

   (b) Integrate the predictor into your pipeline and apply the refinement cycle **only** to patches where the difficulty score exceeds a threshold τ.

3. **Evaluation**
   CMMD (CLIP Mean Maximum Discrepancy) uses CLIP embeddings and Maximum Mean Discrepancy instead of Inception embeddings and Fréchet Distance (FID), offering a more semantically-grounded metric. FID assumes Gaussian feature distributions; CMMD does not, making it more robust to distributional shifts in generative models.

   (a) Implement CMMD evaluation (using CLIP embeddings + MMD) and compare RACD against both the baseline DiT and global cyclic refinement using FID, CMMD, and generation time.

(b) Plot Fidelity (CMMD) vs. Compute Time for varying τ. What is the optimal τ? Why?

Notes:

- Resize all images to 256×256.
- Train latent diffusion models only. Use the pre-trained VAE from the DIT repo; do *not* train in pixel space.
- Fix your DiT model size/config (e.g. DiT-B/8) and use it as the baseline for all experiments.
- Answer all questions in your report and attach generated images from *your* implementation.

BONUS

AdaDiff shows that step selection can be framed as a sequential decision problem. Instead of using a fixed threshold τ, treat the Difficulty Predictor as a *Policy Network* and learn to select the optimal noise-injection level dynamically via Reinforcement Learning.

The Task: For each image patch, the policy network observes intermediate DiT features and outputs a continuous action $t_{start} \in [0, T]$ (the noise injection level) rather than a binary refine/skip decision. Train the policy network using the following reward:

$$\text{Reward} = -(\text{CMMD Error}) - \lambda \cdot (\text{Compute Time})$$

Goal: Train the policy network via RL (e.g. REINFORCE or PPO) to dynamically choose the optimal t_{start} for each patch, learning a smooth trade-off between fidelity and compute. Report CMMD, FID, and compute time, and compare against your RACD baseline with the best fixed τ.

Resources:

- AdaDiff: Adaptive step selection and policy networks
- Region-Aware Sampling (RAS): Spatial heuristics
- Cyclic Diffusion (ABCD): Cyclic noise-injection and denoising mechanics
- DDPM: Maths behind diffusion
- Latent Diffusion Models: Important background read
- CMMD: CLIP-based evaluation metric
- This video covers the core concepts of a diffusion model (highly recommended if this is your first time hearing about diffusion) – watch till 41:40.
- Relevant Papers – DDPM (maths behind diffusion), [VAE], Diffusion models beat gans on image synthesis, Latent Diffusion Models (Important read!).
- Mathematical explanations of DDPMs: [1] [2] [3]
- Additional Videos – KL divergence, explainer for DiT, VAE, VQ-VAE
- Stable Diffusion Explainers – [1] [2]

Recommended Platform: Google Colab, Kaggle Notebook

Mechanistic Interpretability

Large language models encode meaning and context in high-dimensional hidden representations, but efficient deployment often requires low-bit activation quantisation. While quantisation reduces memory and compute costs, it can significantly degrade model performance. In this project, you will investigate how quantisation alters the internal representation space of a Transformer and identify which features and subspaces are most affected. Instead of only measuring perplexity drops, the focus is on understanding why quantisation damages certain representations and whether those representations are actually important for downstream behaviour. We focus on internal representation distortion rather than output degradation alone.

1. **Pipeline Setup.** Use a frozen pretrained LM, `distilgpt2`, and OpenWebText with a target subset of $\sim 1\%$ of the corpus (roughly 80–100M tokens), starting with a 0.2% debug pass. Extract hidden states from *layer 3 of 6* using contiguous sequences of length 128 (avoid padded or isolated-token sampling), then normalize and shuffle hidden states $(batch, seq_len, 768)$ before SAE training to reduce temporal correlation. Train an SAE with top-k sparsity (10%) and bottleneck sizes $m \in \{512, 1024\}$ (start with $m = 512$), using Adam (lr=10^{-4}), batch size 4k–8k, for about 100k steps.

2. **Quantisation Analysis.** Start with per-tensor and per-feature quantisation (add vector quantisation later), and compute ranges from min/max calibration statistics. Sweep bitwidths in staged order (8-bit $\rightarrow$ 4-bit $\rightarrow$ 2-bit, optionally 6-bit), replace *layer 3* activations with reconstructed activations, and continue the forward pass through remaining layers. Evaluate with perplexity (primary; teacher forcing on held-out OpenWebText), SDS, MSE, and CKA (compare original vs reconstructed layer-3 activations on a held-out batch), and visualize geometry with UMAP/t-SNE on $\sim$10k tokens.

3. **Representation Damage and Causal Importance.** Rank the most affected neurons using both activation change (ℓ_2 difference) and KL divergence between full-precision and quantised activation distributions. Run spectral analysis with three required outputs: singular values (before vs after), principal angles, and SDS. For top-ranked neurons, report top-activating tokens, perform ablation (set neuron to zero), measure perplexity change, compare against random neurons, and test alignment between *most quantisation-damaged neurons* and *most perplexity-impacting neurons*.

4. **Mechanistic Explanation and Robust Quantisation.** Investigate why sparsity and quantisation interact destructively by measuring encoder sensitivity using Jacobian norms and Fisher-style importance scores for bottleneck units, and explicitly test low-variance collapse, sparsity fragility, and subspace rotation from quantisation. Based on these findings, implement subspace-preserving quantisation: compute top-k singular vectors of full-precision bottleneck activations, decompose activations into important-subspace and residual components, quantise the important projection at higher precision (e.g., 8-bit) and residuals at lower precision (2–4 bit), then compare against standard quantisation using SDS, perplexity, and representation similarity.

5. **Compute Strategy (Implementation Constraint).** Stream OpenWebText and process in chunks; do not materialize the full dataset or all hidden activations in memory. Store cached activations and use `float16` where stable to control memory and throughput.

BONUS

1. Extend to joint activation and weight quantisation – quantise the frozen LLM weights alongside SAE bottleneck activations and measure the compounded effect on representation damage and downstream perplexity.
2. Replace the SAE bottleneck with a VAE (learning a continuous latent distribution instead of sparse activations), or learn a vector quantiser whose codebook is optimized to explicitly minimize distortion in the important representation subspaces identified in Section 4.

For the VAE option, use

$$\mathcal{L}_{\text{VAE}} = x - \hat{x}_2^2 + \beta \, \text{KL}(q(z \mid x) \, \| \, p(z)) \, .$$

Key Formulas: Document containing the key formulas.

Resources:

- Neel Nanda - Mechanistic Interpretability
- Towards Monosemanticity: Decomposing Language Models With Dictionary Learning
- Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet
- SAE/Mechanistic Interpretability lecture (YouTube)
- SAELens - SAE training library by Joseph Bloom
- TransformerLens - library for extracting hidden states from GPT-style models
- Open-source SAE implementation (reference codebase)

Feel free to reach out for any help or if you need additional resources!

Recommended Platform: Google Colab, Kaggle Notebook

Reinforcement Learning

This section explores whether Transformers' temporal memory gives them an advantage over memoryless policies in online RL, under both full and partial observability.

An RL agent maximises cumulative reward via trial and error. TD Learning updates value estimates after each step via the Bellman equation. Standard RL assumes the Markov property; in practice, environments are often partially observable (POMDPs), requiring integration of past observations (*working memory*).

1. **Implementation.**
   Implement TD3 (MLP actor and critic) on `Hopper-v5` for 1M steps over 3 seeds (*baseline*). Then replace the MLP actor (optionally critic) with a causal Transformer that takes a sliding window of the last L observation-action pairs as input, applies causal self-attention, and outputs a_t. Sweep $L \in \{4, 8, 16, 32\}$ and compare against the baseline.

   Preferred config (others welcome): 2 layers, 4 heads, embed dim 128, L=8, pre-LN, running mean/std normalisation; TD3: lr 3×10^{-4}, buffer 10^6, batch 256, τ=0.005, policy delay 2.

2. **Partial Observability.**
   Apply each modification independently to `Hopper-v5`; compare MLP-TD3 vs. Transformer-TD3 (best L and L=32).

   (a) **Hidden Velocities.** Remove all velocity components; the agent must infer them from consecutive positions.

   (b) **Observation Noise.** Add $\varepsilon \sim \mathcal{N}(0, \sigma^2 I)$; use $\sigma \in \{0.1, 0.3\}$.

   (c) **Delayed Rewards.** $r_t^{\text{delayed}} = \sum_{i=t-K+1}^{t} r_i$ if $t \bmod K = 0$, else 0; use $K \in \{10, 30\}$.

   (d) **RLHF.** Replace environment reward with a learned reward model following Christiano et al. (2017): (i) sample trajectory segment pairs (σ^1, σ^2) from the replay buffer; (ii) label by ground-truth return ($\sigma^1 \succ \sigma^2$ iff $\sum r^{(1)} > \sum r^{(2)}$); (iii) train $\hat{r}_\phi$ via Bradley–Terry cross-entropy $P[\sigma^1 \succ \sigma^2] = \sigma\big(\sum_t \hat{r}(o_t^1, a_t^1) - \sum_t \hat{r}(o_t^2, a_t^2)\big)$; (iv) substitute $\hat{r}_\phi$ for the environment reward during TD3 training. Ground-truth returns are used only to generate labels, not during training. Compare against the ground-truth-reward baseline.

   (e) **Attention Attribution (Optional).** Implement the relevancy propagation of Chefer et al. - propagating relevancy backward through attention layers via partial derivatives, handling residuals and multi-head averaging - as a more principled alternative to raw attention weights.

3. **Attention Analysis.**
   Roll out 5 episodes with trained Transformer agents. Plot mean attention weights over past timesteps per decision; compare fully observable vs. hidden-velocity settings. Track and plot attention entropy $H(A_t) = -\sum_j A_{t,j} \log A_{t,j}$ vs. training steps.

BONUS

(a) **Positional Encoding Ablation.** Compare learned, sinusoidal, and RoPE on the hidden-velocity setting.

(b) **Combined POMDP.** Apply hidden velocities + noise (σ=0.1) + delayed rewards (K=10) simultaneously; report Transformer-TD3, L=32.

(c) **In-Context RL via Algorithm Distillation.** Following Laskin et al. (2022), train a causal Transformer on full cross-episode learning histories (not single trajectories) from a TD3 source policy, so that at test time it improves its policy purely via in-context updates - no gradient step at inference. Key challenges: constructing offline learning-history datasets, correct causal masking across episode boundaries, and evaluating on held-out POMDP tasks.

(d) **xLSTM as Policy Backbone.** Replace the Transformer actor with an xLSTM (Beck et al. 2024) combining sLSTM (scalar memory, exponential gating) and mLSTM (matrix memory, covariance update $C_t = f_t C_{t-1} + i_t v_t k_t^\top$) cells. Implementable in pure PyTorch, but the stabiliser $m_t = \max(\log f_t + m_{t-1}, \log i_t)$ is numerically delicate. Train on hidden-velocity and delayed-reward settings; compare against Transformer-TD3.

Resources:

- David Silver - RL Lecture Series (DeepMind/UCL)
- Andrej Karpathy - Let's build GPT
- Yannic Kilcher - Decision Transformer
- Pieter Abbeel - TD Learning and Q-Learning
- Lilian Weng - A (Long) Peek into Reinforcement Learning
- OpenAI Spinning Up in Deep RL
- Jay Alammar - The Illustrated Transformer
- Hugging Face Deep RL Course

Recommended Platform: Local Machine / Open AI Gym

Sparsity & Optimization

Parameter-efficient fine-tuning (PEFT) methods aim to adapt large pretrained models while updating only a small subset of parameters. One popular approach is Low-Rank Adaptation (LoRA), which assumes that the change required to finetune a model lies in a much lower dimensional subspace. Instead of updating the full weight matrix W, LoRA freezes the pretrained weights W_0 and learns a low-rank update:

$$W = W_0 + BA,$$

where $A \in \mathbb{R}^{r \times d}$ and $B \in \mathbb{R}^{k \times r}$ are trainable matrices and $r \ll \min(k, d)$ is the chosen rank. Several extensions have been proposed to address the limitation of choosing a fixed rank. AdaLoRA dynamically allocates rank during training based on the importance of rank components, while Sparse Low-Rank Adaptation (SoRA) introduces sparsity-controlled gating with ℓ_1 regularization to automatically determine the effective rank.

1. Compare the performance of LoRA, AdaLoRA, and Sparse Low-Rank Adaptation (SoRA) on the CoLA task from the GLUE benchmark (clone the Git repository, and run `python download_glue_data.py --data_dir glue_data --tasks CoLA` to download the dataset) using DeBERTaV3-base as the backbone model.

   Evaluate and compare the methods in terms of:

   (a) Performance on CoLA,

   (b) Number of trainable parameters,

   (c) Effective rank of the adaptation matrices after training,

   (d) Training efficiency in terms of time taken.

   Analyze how adaptive rank selection affects performance and parameter efficiency compared to fixed-rank LoRA.

2. In SoRA, the gating vectors (g) are optimized using proximal gradient descent on the ℓ_1-regularized objective:

$$L(\Delta) = L_0(\Delta) + \lambda \sum_{k=1}^{K} ||g^{(k)}||_1.$$

   The proximal update produces a soft-thresholding step that explicitly drives small gate values to zero, inducing sparsity and reducing the effective rank.

   Investigate what happens if this proximal update is replaced with standard SGD using ℓ_1 subgradients. Specifically:

   (a) Derive the explicit SGD update rule for ($g^{(k)}$), implement it (both forward and backward pass) in both numpy and PyTorch (separately) and test both, comparing them to each other and with tests in part 1.

   (b) Compare it mathematically to the proximal soft-thresholding update.

   (c) Explain whether the choice of subgradient ($\partial ||g^{(k)}||_1$) affects the theoretical comparison between the two methods, where the two methods differ and how.

3. Extend the adaptive low-rank adaptation framework to recurrent sequence architectures such as xLSTM and the Mamba SSM. Investigate how the methodology would need to change to

best fit the sequential nature of these models. Propose an appropriate method as to how low-rank adaptation and SoRA like updates could be applied within xLSTM and Mamba, implement it, and test using the same dataset and metrics as earlier.

BONUS

Implement and explore PEFT methods on the RWKV Language Model. The open ended nature of this task is intentional :).

Note:

- While the math for some stuff might look intimidating (especially Boyd's proximal gradient descent notes), you only really need a small section to be able to solve the questions, so take a look at the resources and it'll seem easier.
- For part 3, the GitHub repositories are [Mamba] and [xLSTM].
- Answer the questions in depth in your report, feel free to add plots, figures and other images (generated by your implementations) to help support your point. The questions requiring math should have the needed steps along with your intuition.

Resources:

- LoRA: Low-Rank Adaptation of Large Language Models
- AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning
- SoRA: Sparse Low-Rank Adaptation of Pre-trained Language Models
- [Video] Proximal Gradient Descent, explained in simpler terms
- [Video] Subgradients explained
- xLSTM: Extended Long Short-Term Memory
- LSTM: colah's blog
- Stanford CS231n Lecture on RNNs and LSTMs (We'd recommend doing this whole course, it's great!)
- Mamba: Linear-Time Sequence Modeling with Selective State Spaces
- Maarten Grootendorst's blog on SSMs (It's a bit long, but should make things clearer through its visualizations.)
- Annotated S4 (should help with SSM implementations!)

Feel free to reach out for any help or if you need additional resources!

Recommended Platform: Google Colab, Kaggle Notebook

Paper Implementation

Your task is to examine the central claim of a paper by implementing it and performing additional empirical experiments.

1. Select any one of the papers published in an A or A* conference recently(NeurIPS, ICML, ICLR, CVPR, AAAI, ACL, etc. [Rankings Portal]). First, read the **abstract** and give a rough reading of the paper you have chosen and identify its central claims. At this point inform us that you are attempting to implement the paper and tell us about the paper chosen.
2. Once you get approval from us, you can go ahead and give it a thorough reading and implement the paper using a framework of your choice (we highly recommend PyTorch).
3. Formulate and perform additional experiments and ablations (such as trying on a different dataset, with different experimental parameters etc...) to test the central claims of the paper. You can discuss these with your mentor.
4. Document your code with a README and a short review of the paper (in LaTeX). The documentation should contain the results of your evaluation and explain how your additional experiments evaluate the paper's claims.

You will be evaluated primarily on your understanding of the paper and the additional evaluations you have performed on its central claims. We are not expecting very clean code, however, it should be able to replicate the results described in the paper and be well-documented. While this task has not been designed for beginners, anyone is welcome to give it a go.